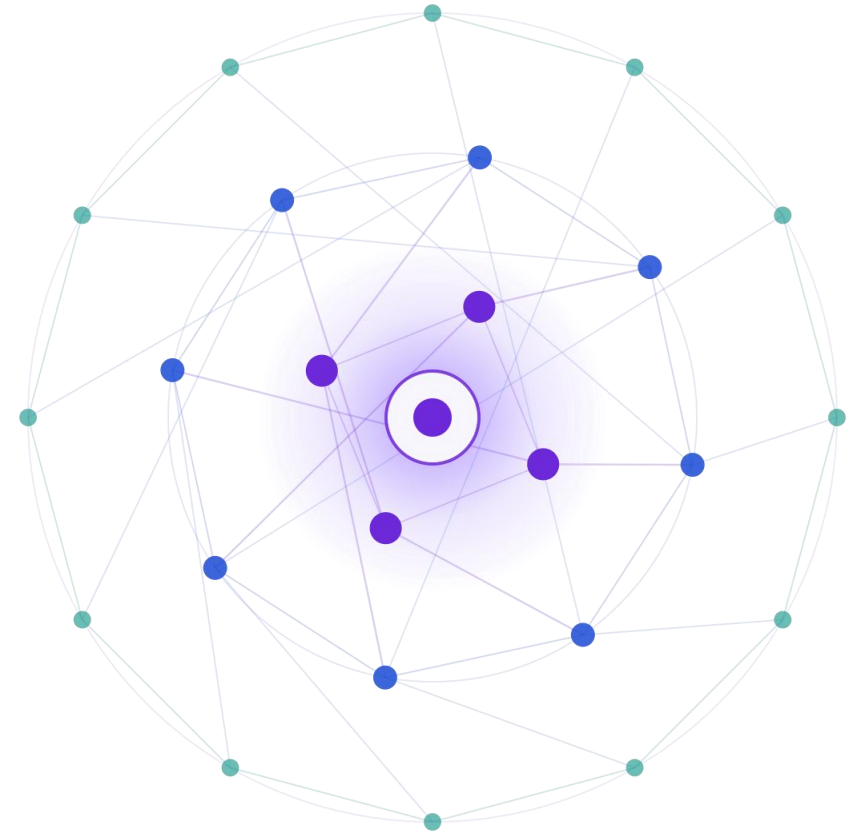


Generative AI Hackathon

3rd Edition 2026 – 27

- Theme ID - 02 · Smart Guided Troubleshooting Engine · Project: FixFlow
- Team Name - SRMIST_QueueQueueing
- College Name - SRM Institute of Science and Technology
- Member Name & Email 1- Abhinav Kumar (Reg. No. RA2411003010993),
ak8045@srmist.edu.in [primary member]
- Member Name & Email 2- Abhishek Verma (Reg. No. RA2411003012011),
av8372@srmist.edu.in
- Member Name & Email 3- Ayushi Paul (Reg. No. RA2411003010997),
ap1547@srmist.edu.in
- Member Name & Email 4- Ketki Gonnade (Reg. No. RA2411003010739),
kg0184@srmist.edu.in
- Submission Github link -
https://github.com/AbhishekVerma295/SRMIST_QueueQueueing_PRISM_Theme02



Theme

Theme 02 · Smart Guided Troubleshooting Engine



“My screen flickers and the battery dies fast.”

- Customers describe symptoms, not Settings screens.
- A support agent reads long knowledge articles, picks the steps that apply and orders them safely, about 15 minutes per scenario.
- The user then hunts through nested Settings menus to act on the advice.
- Our task: turn the complaint (+ the knowledge article) into clean, ordered steps as JSON, each Settings step with an exact one-tap deeplink, served in under 300 ms for known issues.



≈15 min

manual triage per scenario



Millions

of support interactions



10k+

scenarios in production

Existing Solutions & Gaps

Approach today	How it works	Gap
Manual agent triage	Agent reads knowledge articles and writes steps by hand	≈15 min per case; inconsistent order; no deeplinks
FAQ / keyword search	Returns whole articles that match words	User still has to extract steps and find the screens
Generic LLM chatbot	Free-form answer from model memory	Invents steps and web links; wrong or parent-menu screens; not auditable
Rule / decision trees	Hand-built flows per issue	Brittle to new wording; does not scale to 10k+ scenarios
Static deeplink lists	Links per topic	No link between a complaint and the exact screen or toggle



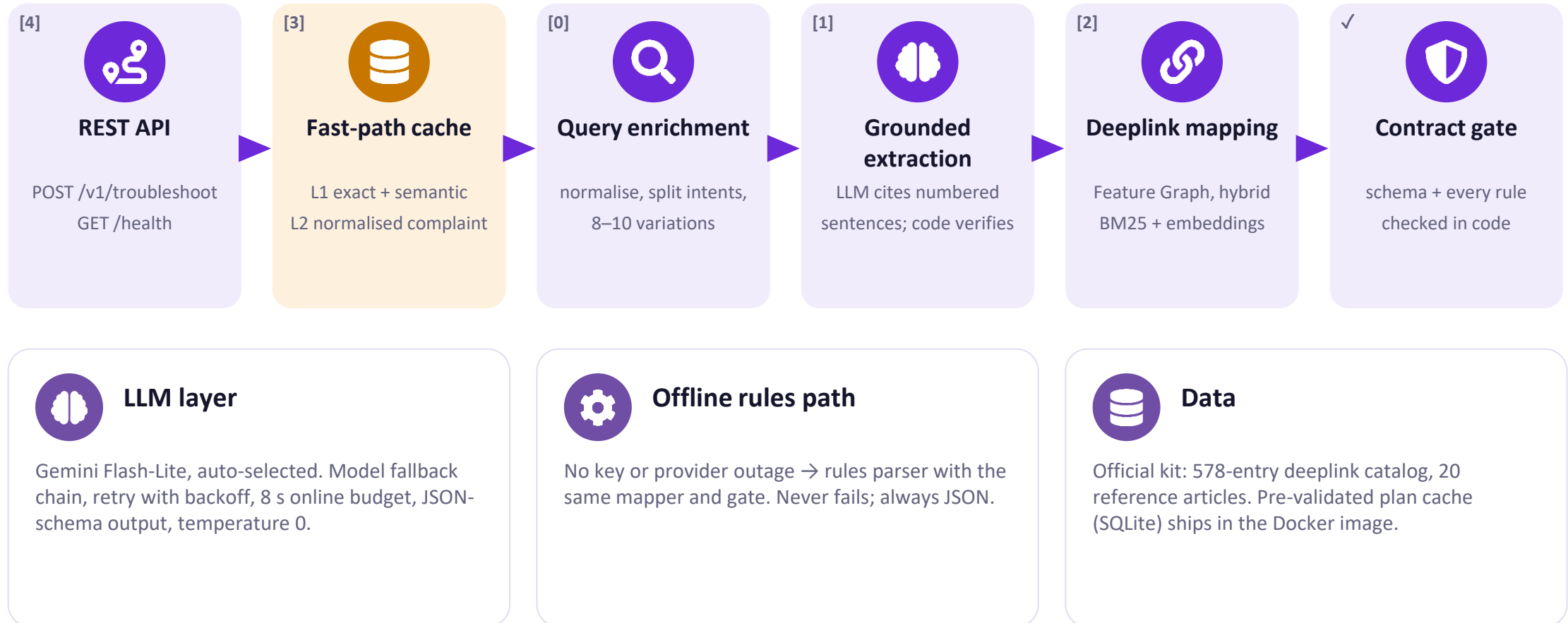
What is missing, and what FixFlow is designed for

Grounded: only steps that exist in the reference article · Exact: the right screen and the right on/off operation from the official catalog

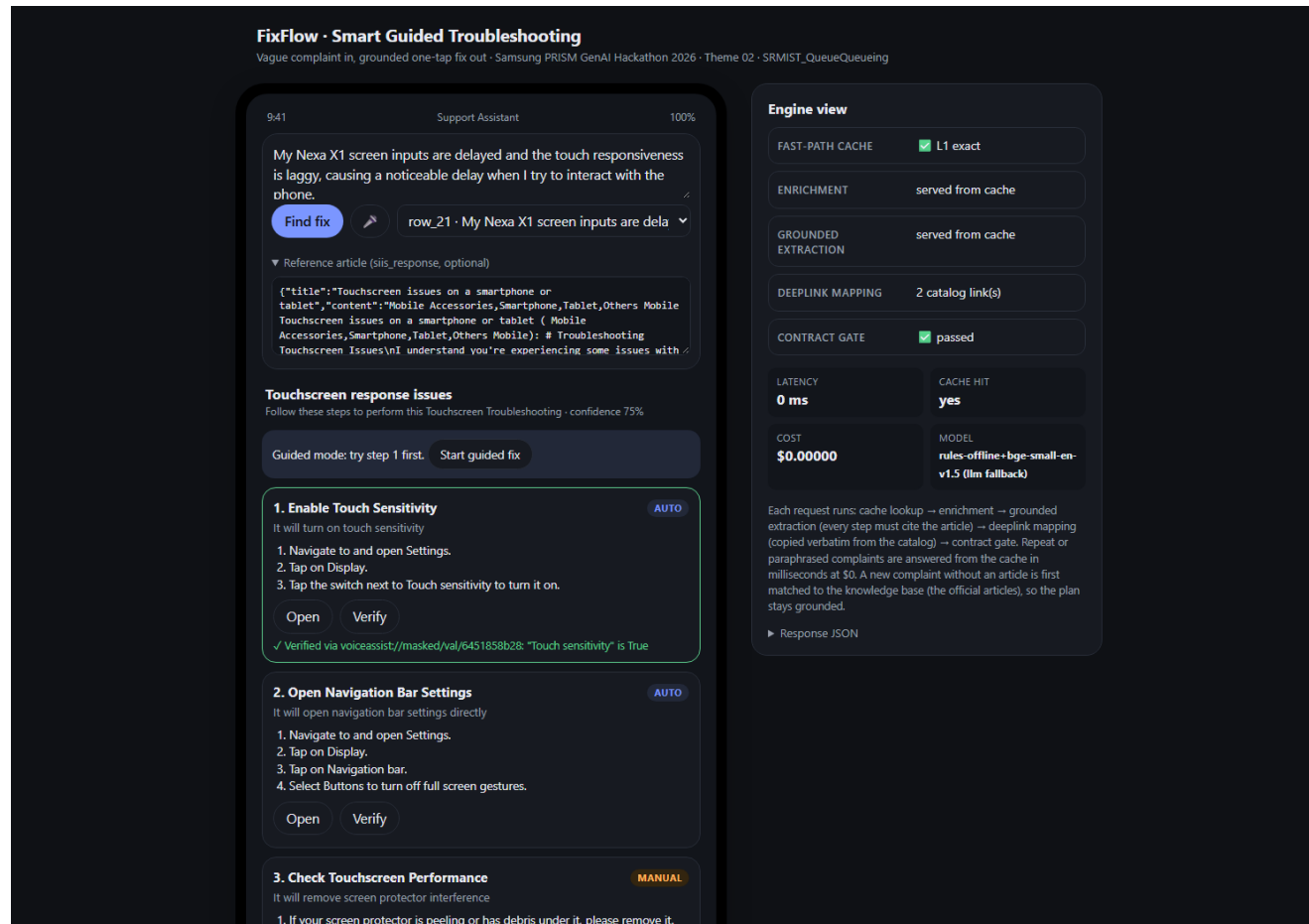
Safe: least disruptive first, restart/reset last · Instant: repeats and paraphrases answered from a validated cache · Verifiable: the fix can be checked

Our Solutions & Architecture Diagram

FixFlow: a REST engine that returns a schema-valid, ordered, deeplinked plan as JSON. The model proposes; code decides.



Demo & Product Walkthrough



Live UI served by the API: phone view (left) and engine view (right)



Describe the problem

Type, speak, or pick one of the 20 official complaints.



Get an ordered plan

Settings toggles → checks → service → restart/reset last.



Tap → Verify

Open applies the setting; the validation deeplink confirms it ("Touch sensitivity is True").



Guided mode

One step at a time: "Fixed?" or "Next".



Engine view

Cache / LLM / rules path, latency, cost, dropped unsupported steps.

Run: docker compose up → localhost:8000

Video: link in README (YouTube, unlisted)

Tools and tech stack used



Python 3.12

FastAPI · Uvicorn · Pydantic v2 (official schema.py, unchanged)



Gemini API

Flash-Lite (3.x) via REST, JSON-schema output, provider-agnostic adapter



Retrieval

fastembed BAAI/bge-small-en-v1.5 (ONNX, CPU) · rank-bm25 · NumPy



Storage

SQLite plan cache with embedded keys · JSON feature graph



Delivery

Docker + docker-compose · model and cache baked in · /health



Quality

pytest (47 hermetic tests) · evaluation harness · ablation · gold labels



Demo UI

Static HTML/JS served at / · Web Speech API for voice input



Built with

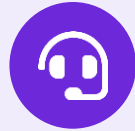
Claude Code as the engineering assistant

Impact & Use case



Customer self-service

Inside a voice assistant or support app: the user says the problem and gets one-tap fixes with verification.



Support-agent assist

The agent pastes the complaint; FixFlow returns an ordered, cited plan in milliseconds instead of ~15 minutes.



Knowledge base at scale

Compile every knowledge article once into validated plans; serve millions of repeats from the cache at \$0.

≈15 min → ms

manual triage vs cached plan

100%

catalog-valid deeplinks (no wrong screen)

\$0

per repeat / paraphrase (cache)

0

invented web links (scrubbed + verified)

Who benefits: customers (faster fixes, fewer service visits), support teams (less handling time, consistent answers), the product team (one reusable mapping for 10k+ scenarios).

Innovation highlights, results and limitations

Innovation

- Citation-verified extraction: code rejects steps not in the article
- Action- and depth-aware deeplinks (exact screen, right operation)
- Closed-loop Tap → Verify with validation deeplinks
- Trust-tiered, symptom-guarded two-level cache
- Outage-proof: model fallback chain + offline rules

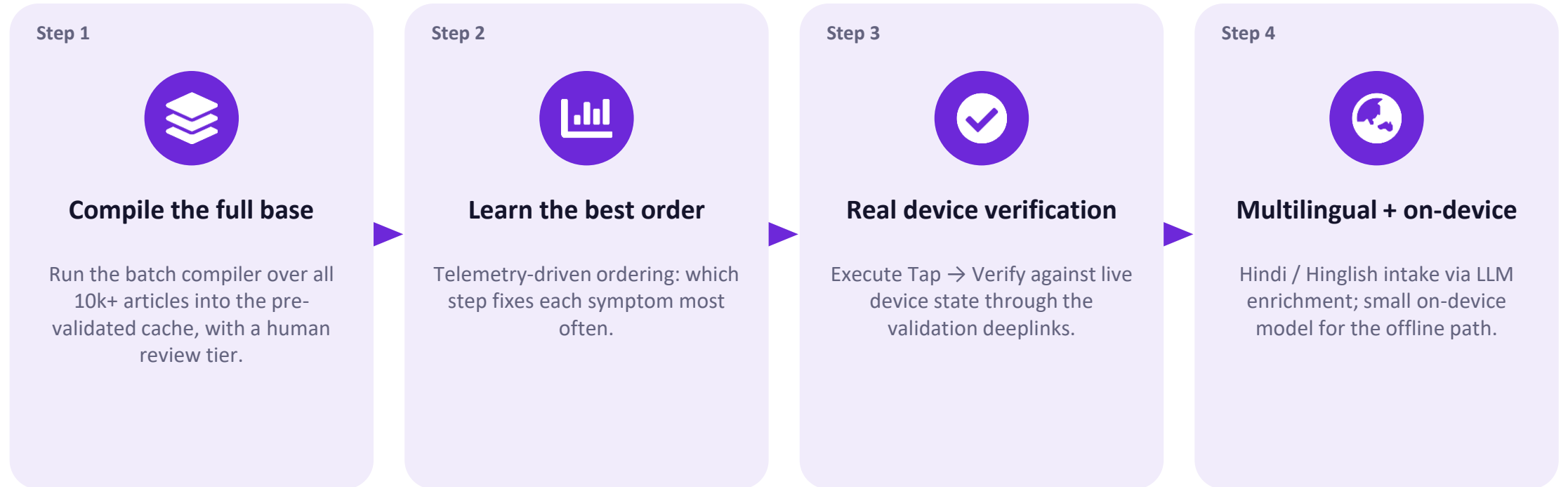
Results

- Gates 100%: schema, rules, 0 URL leaks, catalog-valid links (D1 + D2)
- Deeplink precision 100%; relevance 2.00 (D1) / 1.71 (D2) of 2
- Step-accuracy proxy 2.20 (D1) / 2.88 (D2) of 3; abstention 90% / 100%
- Cache P95: 0 ms exact, 48 ms unseen paraphrase; 81.8% hit
- Ablation: hybrid 1.76 rel / 100% precision vs rules 1.29 / 58%

Limitations

- Provider outage: the LLM built 12 of 20 official plans; the rest used rules
- Paraphrases: every hit had the right symptom; 59% the exact same article
- Offline relevance is coarse; LLM sometimes drops a section
- Gold labels by one annotator; D2 used once for error analysis
- Contract puts service escalation before restarts

What's next



Worklet fit: the engine, evaluation harness and batch compiler already exist; the next step is scale and live-device integration, not a rewrite.

Brownie points slide(differentiation)



Measured, not claimed

Held-out paraphrases, 4-domain scenarios, gold labels, and a scripted 3-way ablation, with honest limits.



Outage-proof, proven

During a real Gemini 503 outage, 30 of 32 cold requests fell back to rules and every one stayed contract-valid.



Closes the loop

Validation deeplinks confirm the fix actually happened, not just a link to a screen.



Contract-exact

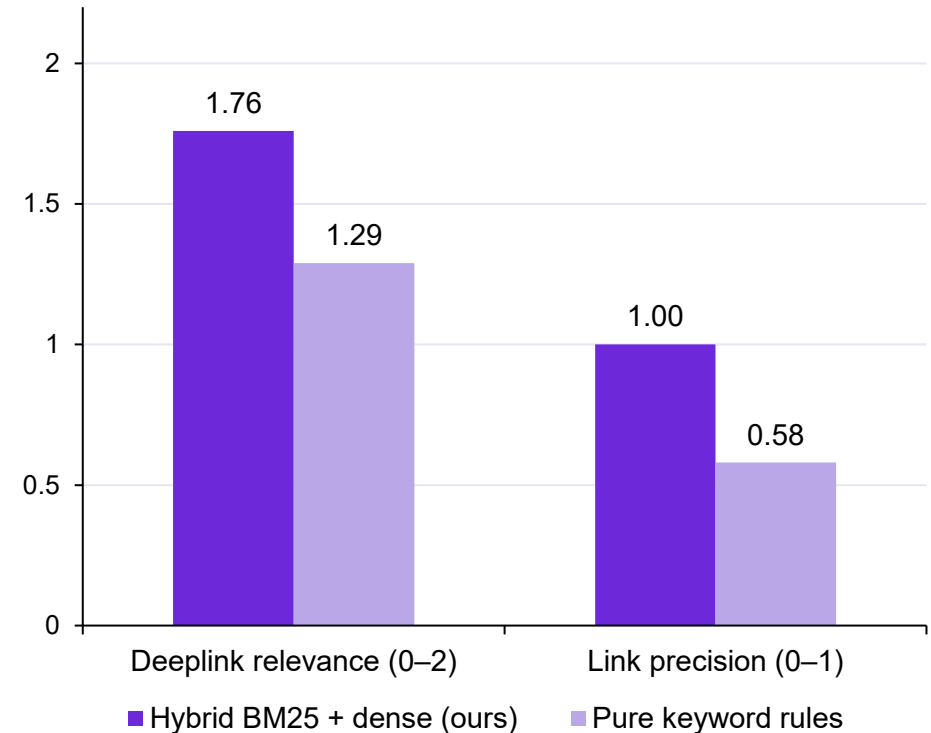
Official schema unchanged, every rule enforced in code, brand-neutral outputs, 47 hermetic tests.



Zero-friction to judge

docker compose up: model and validated cache baked in; runs with no API key.

Ablation: same extraction, different mapper



Full-LLM mapping baseline implemented (eval/ablation.py); not completed due to the provider outage.

Checklist- Updated on Public GitHub

- Working prototype code — public GitHub repo: Y
(github.com/AbhishekVerma295/SRMIST_QueueQueueing_PRISM_Theme02, tag `PRISM_GENAI_HACKATHON_Y2026`)
- README with reproducible setup instructions: Y (docker compose up, or Python 3.12 + 3 commands)
- Demo video, max 5 minutes: link in README (YouTube, unlisted)
- Presentation file (PPT or PDF): Y (docs/SRMIST_QueueQueueing.pptx and .pdf)



Everything referenced here is inside the tagged commit `PRISM_GENAI_HACKATHON_Y2026`: code, README, Dockerfile, results.jsonl, metrics.md, this deck (PPTX + PDF) and the video link.

Thank you

FixFlow · Theme 02 · SRMIST_QueueQueueing

Abhinav Kumar · Abhishek Verma · Ayushi Paul · Ketki Gonnade

Repository: github.com/AbhishekVerma295/SRMIST_QueueQueueing_PRISM_Theme02 **Tag:** PRISM_GENAI_HACKATHON_Y2026